IntelliJ/Eclipse + Ant: PRIME v2 – Local Automation Setup

Overview:

This guide provides step-by-step instructions for setting up IntelliJ IDEA to work with the PRIME v2 automation framework. PRIME v2 uses Ant for building and requires specific environment variables and JAR dependencies to be configured manually.

> ***Scope**: Prime **version 2** local setup for **Automation** projects (separate from API/common libs). v2 uses **Ant**, **Java 8**, and **manual JAR dependencies** (no Maven). This guide covers prerequisites, environment variables, cloning (GitLab/CLI /SourceTree), IDE setup, adding JARs, Ant builds, runners (host.xml), and FAQs.*

What's different in v2 vs v3

- v2 is **Ant + Java 8**; requires local folders for **ant, cucumber, selenium, testng, groovy**, + PRIME jars.
- You must set **system environment variables** (e.g., `ANT_HOME`, `SELENIUM_HOME`, `PRIME_HOME`, `JAVA_HOME`).
- Dependencies are **added as JARs/dirs** in your IDE (Project Structure), not via pom.
- Build via **Ant targets** (`build_prime`, default `ant`, `cleanbuild`) and run via a **TestNG XML runner** (e.g., `host.xml`).

1) Prerequisites

Required Tools

Tool	Requirements	Setup Instructions
Java 8	JDK 1.8 or compatible	• Download and install from Oracle • Set `JAVA_HOME` to installation path (e.g., `C:\Program Files\Java\jdk1.8.0_40`) • Add `%JAVA_HOME%\bin` to system Path
Ant	Version 1.9.4 or later	• Copy from shared drive: `L:\Departments\Financial Services\UII\UII\UII-QA\Cucumber_Automation\automation-env\ant` • Recommended location: `C:\ant\ant-1.9.4`
Groovy	Version 2.4.11	• Copy from shared drive • Recommended location: `C:\groovy\groovy-2.4.11`
IntelliJ IDEA	Community or Ultimate	• Download from JetBrains website • Ensure 32/64-bit matches Java version
Git	For version control	• Git (for CLI/SourceTree) and IntelliJ IDEA (or Eclipse if you prefer legacy setup).
Other	Environment Variables	◦ Local folders with the following (latest versions from your team share): ▪ `ant/` · `cucumber/` · `selenium/` (includes `startSeleniumGrid.bat`) · `testng/` · `groovy/` · `PRIME libs & config (to include prime.jar)`

2) Environment Variables Setup

Step 1: Copy Required JARs

Copy the newest versions from the shared drive to your local machine (C:\ recommended): `L:\Departments\Financial Services\UII\UII\UII-QA\Cucumber_Automation\automation-env`

Required components:

- `ant` - Build tool
- `cucumber` - BDD framework
- `selenium` - Including `startSeleniumGrid.bat`
- `testng` - Test framework
- `groovy` - Scripting support (For Vespa)

Step 2: Configure System Environment Variables

Go to **Control Panel > System > Advanced system settings > Environment Variables** and add:

Variable	Value	Example
ANT_HOME	Path to Ant installation	C:\ant\ant-1.9.4
CUCUMBER_HOME	Path to Cucumber	C:\cucumber\4.2.0
SELENIUM_HOME	Path to Selenium	C:\selenium\selenium-3.141.59
TESTNG_HOME	Path to TestNG	C:\testng\testng-6.8
GROOVY_HOME	Path to Groovy	C:\groovy\groovy-2.4.11
PRIME_HOME	Path to your project folder	C:\work\unite-aug\automation\unite-test-automation\unite *Note: multiple runners can be added with "," separated*
JAVA_HOME	Path to Java 8 installation	C:\Program Files\Java\jdk1.8.0_40

Step 3: Update Path Variable

Add the following to the beginning of your system Path variable:

`%JAVA_HOME%\bin;%ANT_HOME%\bin;%SELENIUM_HOME%`

Important: Remove any existing references to java/ant/selenium to avoid conflicts.

> *Note: After updating variables, **restart** terminals/IDE so the changes apply.*

Step 4: Verify Environment Setup

a. Open a new command prompt (restart if already open)
b. Run the `set` command
c. Verify all paths are resolved correctly:

cmd

```
echo %JAVA_HOME%
echo %ANT_HOME%
echo %SELENIUM_HOME%
java -version
ant -version
```

3) Chrome Driver Setup

Download Compatible Chrome Driver

a. Check your Chrome version: **chrome://settings/help**
b. Download matching driver from: Chrome-Driver downloads
c. Place the driver in your `SELENIUM_HOME` directory
d. The driver will be picked up automatically via the Path variable

4) Clone/Pull Code from Repository

Gitlab Repo: https://gitlab.com/ascensus-gs/products/depot/qa-automation/automation.git

A) IntelliJ – *Get from Version Control*

a. Open IntelliJ **Get from VCS**.
b. Paste the GitLab repo URL choose a local folder **Clone**.
c. Authenticate with GitLab **PAT/SSO** when prompted.

B) Git CLI

```
cd <your workspace>
git clone <gitlab repo url>
cd automation
```

C) SourceTree

a. **Clone** paste the repo URL choose destination **Clone**.

5) Import/open the project in the IDE

IntelliJ (recommended) - For Unite project

a. **File Open** select the root automation folder (e.g., `unite-test-automation\unite`).
b. **Project SDK**: *File Project Structure Project Settings Project* set SDK to **Java 8**; Language level **8**.
c. **Add library folders (JARs)**: *Project Structure Project Settings Modules Dependencies* **+ JARs or directories...**

- Add these **folders** (select "**Classes**"):
  - `<PRIME root>\prime\lib`
  - `<PRIME root>\prime\config` (PRIME framework jars)
  - `<CUCUMBER_HOME>\lib` (and jars inside)
  - `<SELENIUM_HOME>\lib` (and WebDriver jars)
  - `<TESTNG_HOME>\lib`
  - `<GROOVY_HOME>\lib`
  - JDK tools: `<JAVA_HOME>\lib\tools.jar` (if needed)

d. Install plugins (optional): **Cucumber for Java**; **TestNG**.

IntelliJ (recommended) - For Automation project (Complete V2 Project)

a. **File Open** select the root automation folder (e.g., `automation`).
b. **Project SDK**: *File Project Structure Project Settings Project* set SDK to **Java 8**; Language level **8**.
c. **Add library folders (JARs)**: *File Project Structure Project Settings Modules Dependencies* **+ JARs or directories...**

- Add these **folders** (select "**Classes**"):
  - `<PRIME root>\prime\lib`
  - `<PRIME root>\prime\config` (PRIME framework jars)
  - `<CUCUMBER_HOME>\lib` (and jars inside)
  - `<SELENIUM_HOME>\lib` (and WebDriver jars)
  - `<TESTNG_HOME>\lib`
  - `<GROOVY_HOME>\lib`
  - JDK tools: `<JAVA_HOME>\lib\tools.jar` (*if needed, for Javadoc support*)

Set Project Encoding

a. Right-click on `automation` project
b. Go to **Properties > Resource**
c. Set **Text file encoding** to `UTF-8`

Configure Ant View

a. **Window > Show View > Ant** (or check under Other)
b. Add build files by clicking the **+** icon:
   - `[project]/build.xml` (for compiling)
   - `[project]/bin/build.xml` (for running)
c. Repeat for each project (astro/jett/unite) you're working with

Step 5: Setup Version Control

For Git:

a. **File > Settings > Version Control > Git**
b. Configure Git executable path
c. Setup remote repository

Step 6: Install Cucumber Plugin (Optional but Recommended)

a. **File > Settings > Plugins > Marketplace**
b. Search for "Cucumber for Java"
c. Install and restart IntelliJ
d. This enables navigation between feature files and step definitions

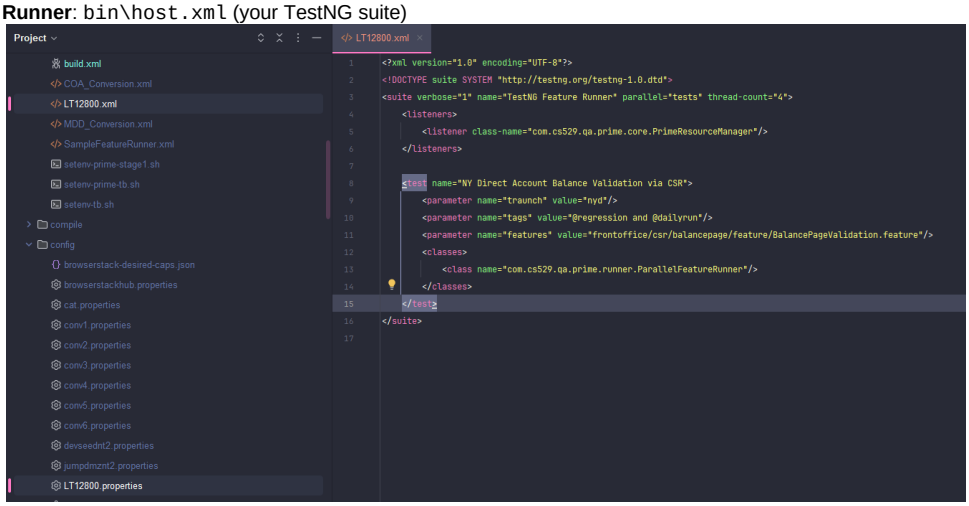
> ***Note:*** *May need to be on WiFi (off corporate network) for marketplace access*

Eclipse (legacy)

- Create a **Java Project** named `automation`, point to the existing source folder.
- Add the same external **JARs** to **Java Build Path** (Libraries tab).
- Add Ant views: Window Show View **Ant**; then add `build.xml` scripts described below.

6) Personal configuration files

- In your project's `config/` folder, copy `sample.properties` rename to **`<COMPUTERNAME>.properties`**. e.g. `LT12800.properties`
- Set the testbed ID and any local paths:
  - `TBID=tb1` this selects `tb1.properties` at runtime.
  - Default Reporter base: `C:\Reporter` (reports are under `YYYYMMDD/<RunnerName>`).
- For this project we also use **host files**:
  - **Runner**: `bin\host.xml` (your TestNG suite)



  - **Properties**: `config\host.properties` (environment + personal overrides)

properties

```
# Database Configuration
TBID=stage1
UNITEDATABASEURL=localhost:41521:UIIS01
UNITEUSERNAME=<DB username>
UNITEPASSWORD=<your password>

# Browser Settings
BROWSER=chrome
WAITTIME=1000
IMPLICITWAITTIME=30
HEADLESS=FALSE
FEEDWAITTIME=60000
SLEEPTIME=1000

# Environment Specific
ENVIRONMENT=testbed
PROJECT=unite
```

> *When missing, the framework falls back to default config files; use `host.properties` to keep local overrides out of source control.*

7) Ant scripts you'll use

Each application repo (ASTRO/UNITE/JETT) contains two relevant Ant files:

- **`<app>\build.xml`** – **compile** targets (`build_prime`, default `ant`, `cleanbuild`).
- **`<app>\bin\build.xml`** – **run** targets (your user/host target that launches TestNG with your suite XML).

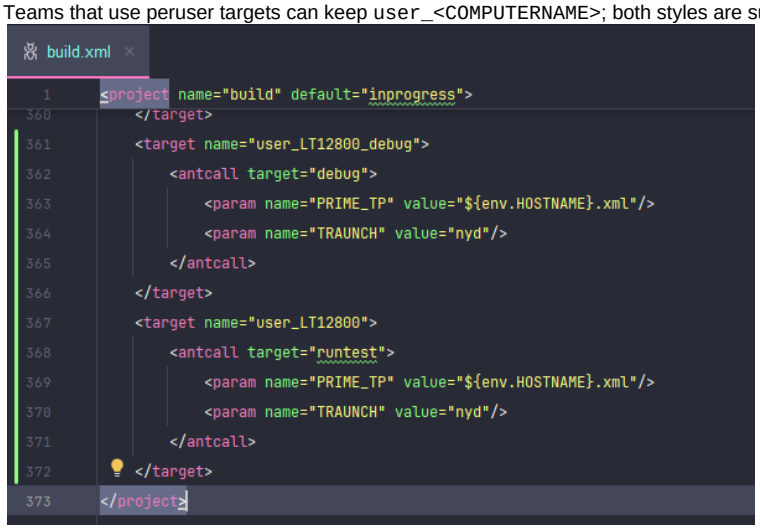
Add your host target (run config)

- Open <app>\bin\build.xml.
- Add a target that points to your runner (example):

```
<!-- Your Name -->
    <target name="user_[COMPUTERNAME]">
        <antcall target="debug">
            <param name="PRIME_TP" value="[HOSTNAME.xml]"/>
            <param name="TRAUNCH" value="[traunch_prefix]"/>
        </antcall>
    </target>

Note: target can be debug/runtest
```

- Teams that use peruser targets can keep user_<COMPUTERNAME>; both styles are supported.



(If requested) Add a host target for compile file

- Some teams also add a convenience host target to <app>\build.xml that just depends on cleanbuild.

8) Building the Project

> *Build from the **application root** (e.g., unite\). The prime folder must be at the same level for build_prime to work.*

- **Compile/Build PRIME only** (needed if PRIME changed):
  - cd \automation\prime
  - ant cleanup-builddir
- **Clean build app + PRIME** (common on first run or after updates):
  - cd <project_folder>
  - ant cleanbuild
- **Compile app only** (ASTRO/UNITE/JETT):
  - cd <project_folder>
  - ant

> *After a successful build, <app>\lib contains **prime.jar**, PRIME libs, and the app's *-automation.jar.*

9) Run tests

Two common patterns are used; use the one your repo adopts.

Pattern A – via bin Ant runner

```
cd <app>\bin
ant
```

```
Note: runs TestNG with bin\host.xml
```

This will:

a. Compile the project (if needed)
b. Run tests using your personal configuration
c. Generate reports in `C:/Reporter/YYYYMMDD/[Runner Name]`

Notes

- **Feature path** in your runner should be **relative to ****PRIME_HOME**.
- Control verbosity (`testng.verbose`) (1-10) and parallelism (`thread-count`) in your suite XML.
- Don't change the framework runner class (Cucumber/TestNG integration is fixed in PRIME v2).

Setup Remote Debugging

a. Edit `bin/build.xml` - change your target from `runtest` to `debug`
b. In IntelliJ:
    - **Run > Edit Configurations**
    - Add new **Remote JVM Debug**
    - Host: `localhost`
    - Port: `8888`
c. Start test execution normally with `ant`
d. Execution will pause waiting for debugger
e. In IntelliJ, run your debug configuration to connect

10) Selenium Grid (local)

1. Run `<SELENIUM_HOME>\startSeleniumGrid.bat` (two consoles will open; keep them open).
2. Verify grid at `http://localhost:4444/grid/console`.
3. If Chrome/Driver mismatch occurs, download the matching **ChromeDriver** and replace the one under `SELENIUM_HOME`.

11) IntelliJ tips for v2

- If classes are **red** after import, you likely missed a JAR folder. Readd via *Project Structure Modules Dependencies*.
- Keep **Java 8** for both **Project SDK** and **Module SDK**.
- Disable Maven autoimport (not used for v2).

12) Frequently Asked Questions (FAQ)

Q1) `ant` is not recognized / Ant fails to start.
A: Ensure `ANT_HOME` is set and `%ANT_HOME%\bin` is added **before** any other Ant path in `PATH`. Restart your terminal/IDE.

Q2) "Could not find or load main class" when running host.xml.
A: Missing JARs on the classpath. Confirm that PRIME libs **and** cucumber/selenium/testng/groovy folders are added as **dependencies** (as directories) in your IDE and are included by the Ant `classpath`.

Q3) ChromeDriver version mismatch.
A: Replace the driver under `SELENIUM_HOME` with a version that matches your Chrome. Download link

Q4) Reports/screenshots are not where I expect.
A: Default base is `C:\Reporter`. Adjust in your `<COMPUTERNAME>.properties` or host properties.

Q5) My feature path isn't being found.
A: In v2, TestNG suite `features` paths are typically **relative to ****PRIME_HOME**. Doublecheck the path and that `PRIME_HOME` points to your app root.

Q6) How do I run a different traunch?
A: Pass a different `traunch` value via your Ant run target (or inside your suite XML parameter). Keep it lowercase (e.g., `ors`, `ils`).

Q7) How do I debug locally?
A: Create an Ant `debug` target that starts the JVM in debug mode (e.g., port 8888), then attach **IntelliJ/Eclipse** Remote Debugger and run your normal `host` target.

Q8) Do I need to rebuild PRIME every time?
A: No—only when PRIME code changes. Otherwise a standard `ant` on the app is sufficient.

13) Troubleshooting

Common Issues and Solutions

Issue	Solution
"JAVA_HOME not set"	Restart command prompt/IntelliJ after setting environment variables
Compilation errors	Verify all JAR dependencies are added to project structure
"Cannot find Chrome driver"	Ensure driver version matches Chrome browser version
Selenium Grid not starting	Check if ports 4444/5555 are already in use
Feature file not found	Verify path is relative to PRIME_HOME
TestNG not running	Check that testng.jar is in classpath
Perforce connection failed	Verify network access to herring:1818

14) Quick Reference

Quick Reference - File Locations

File Type	Location
Build files	`[project]/build.xml`, `[project]/bin/build.xml`
Configuration	`[project]/config/[COMPUTERNAME].properties`
TestNG Runner	`[project]/bin/[COMPUTERNAME].xml`
Feature files	`[project]/features/`
Step definitions	`[project]/src/`
SQL queries	`[project]/sql/`
Error messages	`[project]/errors/`
Element locators	`[project]/element_locators/`
Test reports	`C:/Reporter/YYYYMMDD/[Runner Name]/`

Environment

```
Set ANT_HOME, CUCUMBER_HOME, SELENIUM_HOME, TESTNG_HOME, GROOVY_HOME, PRIME_HOME, JAVA_HOME
PATH += %JAVA_HOME%\bin;%ANT_HOME%\bin;%SELENIUM_HOME%
```

Clone: IntelliJ (Get from VCS) • CLI `git clone` • SourceTree

IDE: Add JAR **folders**: prime/lib, cucumber/lib, selenium/lib, testng/lib, groovy/lib, and JDK tools.jar

Build

```
cd <app>
ant cleanbuild   # or: ant, ant build_prime
```

Run

```
cd <app>\bin
ant
```

Verification Commands

cmd

```
# Check Java
java -version

# Check Ant
ant -version

# Check environment variables
echo %PRIME_HOME%
echo %ANT_HOME%
echo %SELENIUM_HOME%

# List project JARs
dir [project]\lib
```

15) SDET Syllabus (v2)

Day 1–2: Environment variables, clone/import, add JARs, Grid check.

Day 3–4: Build & run with `host.xml`; explore `bin/` and `config/` (`host.properties`, `tb*.properties`).

Day 5–7: Create/edit a small feature + CSV; wire into your host suite; validate reports in `C:\Reporter`.

Stretch: Add a personal `debug` target; practice switching traunch/environment quickly.

Best Practices

1. **Always restart** command prompts and IntelliJ after changing environment variables
2. **Never modify** PRIME core code - it's shared across all projects
3. **Use your COMPUTERNAME** for all personal configuration files
4. **Keep Selenium Grid running** during test execution
5. **Pull latest code** before starting work each day
6. **Clean build** when switching between projects or after major updates
7. **Document test data** used in `QA_UNITE_TEST_CASES` or `QA_SSRP_TEST_CASES` tables

Appendix: Where files typically live

- **PRIME framework**: `...depot\qa-automation\automation\prime`
- **App repos**: `...\automation\unite-test-automation\unite` (or `astro`, `jett`)
- **Runners**: `<app>\bin\host.xml` (or `COMPUTERNAME.xml` conventions)
- **Properties**: `<app>\config\host.properties`, `<app>\config\tb1.properties` etc.
- **Reports**: `C:\Reporter\YYYYMMDD\<RunnerName>`

For additional help, refer to:

- Wiki: http://qawiki.int.uprinv.com/qawiki/index.php/QA_Automation_Framework